

Multi-Object Synchronization

Main Points

- Problems with synchronizing multiple objects
- Definition of deadlock
 - Circular waiting for resources
- Conditions for its occurrence
- Solutions for avoiding and breaking deadlock

Large Programs

- What happens when we try to synchronize across multiple objects in a large program?
 - Each object with its own lock, condition variables
 - Is concurrency modular?
- Deadlock
- Performance
- Semantics/correctness

Deadlock Definitions

- Resource: any (passive) thing needed by a thread to do its job (CPU, disk space, memory, lock)
 - Preemptable: can be taken away by OS (and later given back)
 - Non-preemptable: must leave with thread
- Starvation: thread waits indefinitely
- Deadlock: circular waiting for resources
 - Deadlock $\Rightarrow$ starvation, but not vice versa

Assumptions

We consider re-usable resources

- example: critical sections, printers, ...

- Request

- example: `lock.acquire`, `cond.wait(&lock)`, ...

- hypothesis: *blocking request*

- Assignment and use

- If need to wait, *keep the resource*, i.e., *wait while holding* the resource (non-preemptable resource)

- Release

- example: `lock.release`, `cond.signal()`, ...

Example: two locks

Thread A

```
lock1.acquire();
```

...

```
lock2.acquire();
```

```
lock2.release();
```

```
lock1.release();
```

Thread B

```
lock2.acquire();
```

...

```
lock1.acquire();
```

```
lock1.release();
```

```
lock2.release();
```

Two locks and a condition variable

Thread A

```
lock1.acquire();  
...  
lock2.acquire();  
while (need to wait)  
    condition.wait(lock2);  
lock2.release();  
...  
lock1.release();
```

Thread B

```
lock1.acquire();  
...  
lock2.acquire();  
....  
condition.signal();  
lock2.release();  
...  
lock1.release();
```

Bidirectional Bounded Buffer

Thread A

```
buffer1.put(data);  
buffer1.put(data);
```

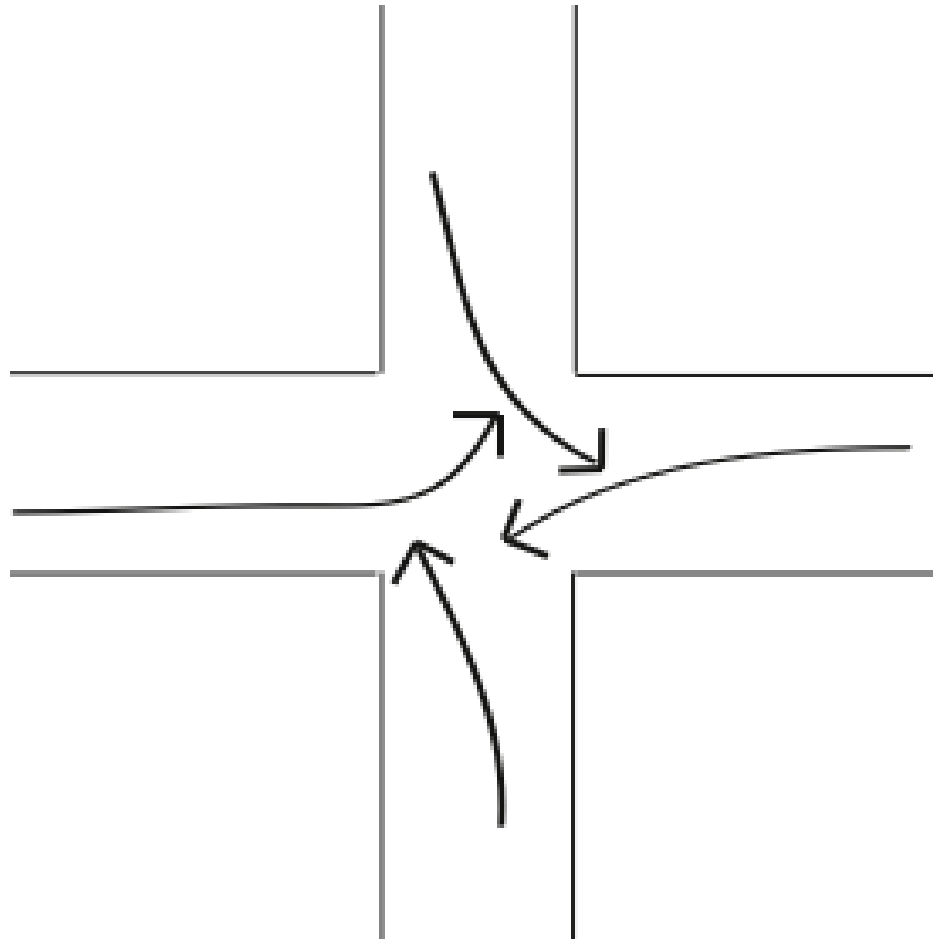
```
buffer2.get();  
buffer2.get();
```

Thread B

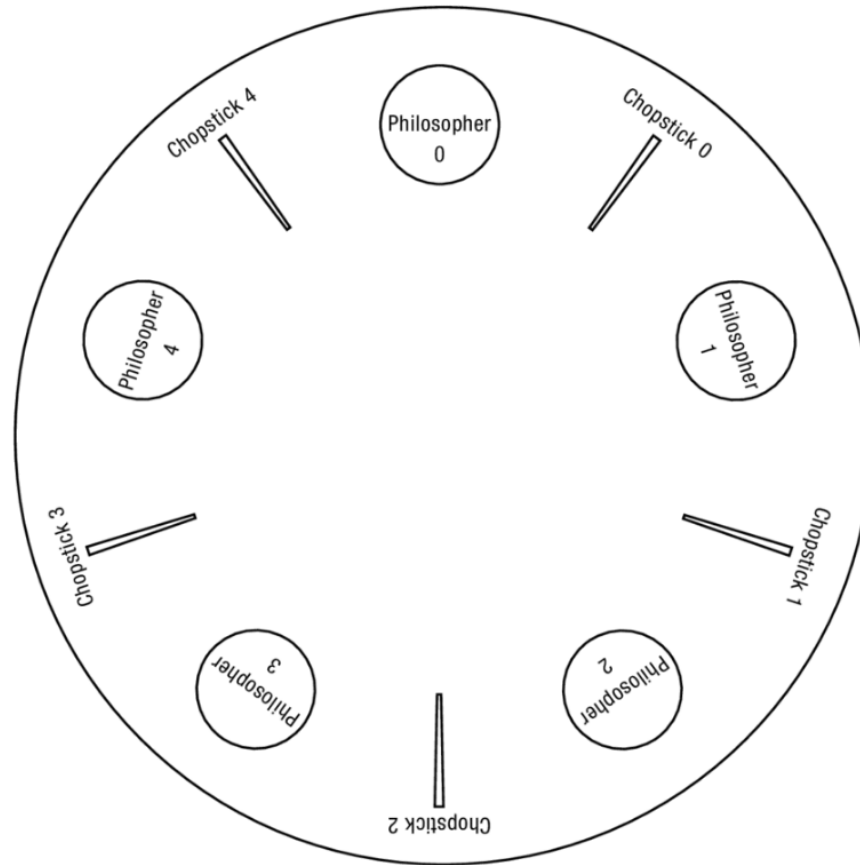
```
buffer2.put(data);  
buffer2.put(data);
```

```
buffer1.get();  
buffer1.get();
```

Yet another Example



Classic Problem: Dining Philosophers

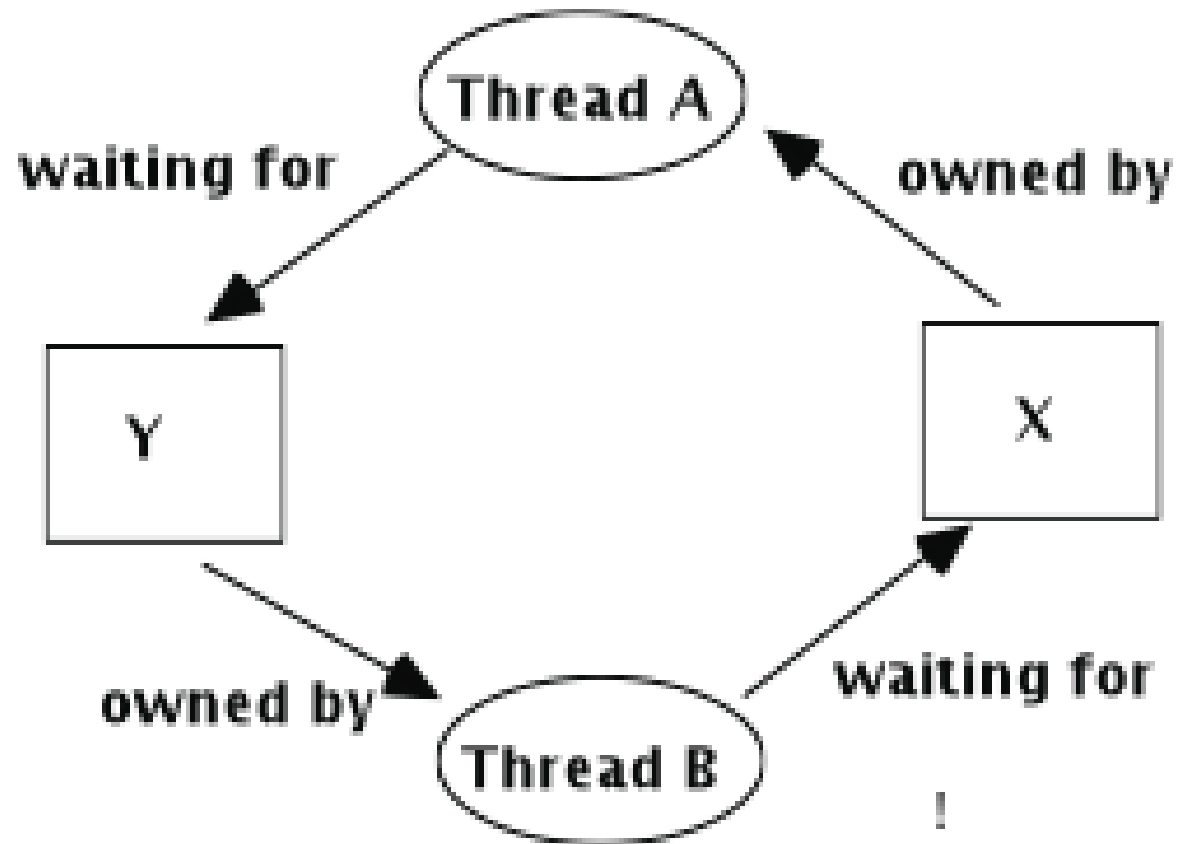


Each philosopher needs two chopsticks to eat.
Each grabs chopstick on the right first.

Conditions for Deadlock

- Limited access to resources
 - A limited number of threads can use simultaneously a resource
 - If at most one: mutual exclusion
 - If “infinite” resources (virtualized), no deadlock!
- No preemption
 - If resources are virtual, can break deadlock
- Wait while holding
 - A thread holds the assigned resources while waiting for another one (aka multiple independent requests)
- Circular chain of requests

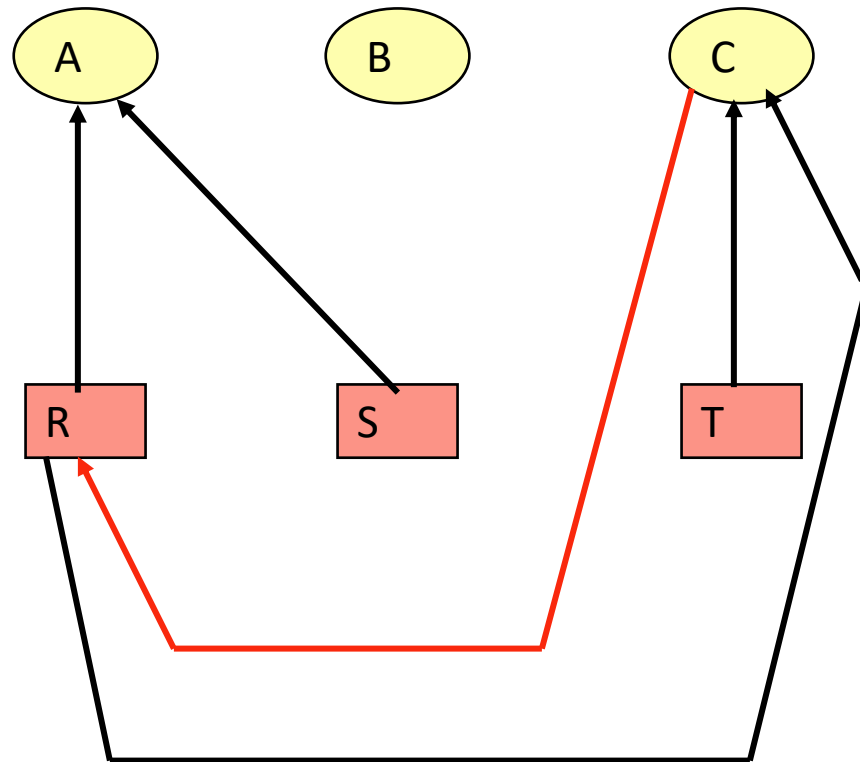
Circular Waiting



Example: sequence NOT leading to a deadlock

A requests R
C requests T
A requests S
C requests R
A releases R
A releases S
...

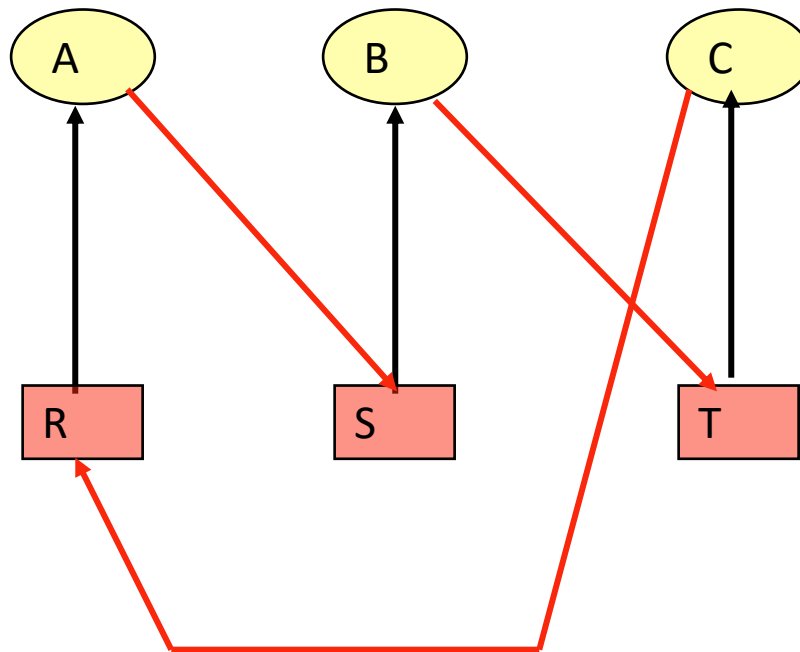
hyp: A needs only R and S
C needs only T and R



Example: sequence leading to a deadlock

A requests R
B requests S
C requests T
A requests S
B requests T
C requests R

Deadlock!



Methods to deal with deadlock

- Detect and fix
- Static prevention
- Dynamic prevention (banker's algorithm)

In practice many systems do not adopt any solution

- The case of Unix, Windows,...
- Make sense if:
 - The cost for prevention or deadlock fixing is too high
 - The system is plentiful of resources (and the chance for a deadlock is very low)

Solution #1: Detect and Fix

- Algorithm
 - Scan the graph
 - Detect cycles
 - Fix cycles
- How?
 - Remove one thread, reassign its resources
 - Requires exception handling code to be very robust
 - Roll back actions of one thread
 - Databases: all actions are provisional until committed

Solution #1: Detect and Fix

- Thread suppression
 - Very simple, rather rough
 - The resources assigned to the suppressed thread can be re-assigned
 - Needs for criteria for the selection of the thread
 - “minimum impact”

Solution #1: Detect and Fix

- Roll-back actions:
 - Restore a previous state of the processes
 - The processes start again
 - Luckily, the deadlock will not appear again
- Requires check-pointing
 - Periodic storage of the internal state of the processes
 - Memory, registers, files,...
- No surprise OSs generally prefer thread suppression

Solution #2: Deadlock Prevention

Eliminate one of the four conditions for deadlock

- Lock ordering (avoids circular waiting)
 - Always acquire locks in the same order
 - Example: acquire resources in alphabetical order (always!!)
 - Widely used in OS kernels
- Design system to release resources and retry if need to wait
 - No “wait while holding”
 - Example: telephone circuit setup. If busy, retry!
- Infinite resources? Not always possible!
- Acquire all needed resources in advance
 - No “wait while holding”
 - Ex: UNIX reserves a process for the sysadmin to run “kill”

Solution #2: Deadlock Prevention

No “wait while holding”

- A process should know in advance all the resources it needs
- It requests in advance the resources
 - If not available -> wait *without holding*!
- Drawbacks
 - Need to know many things...
 - Locks resources that can be used by other processes

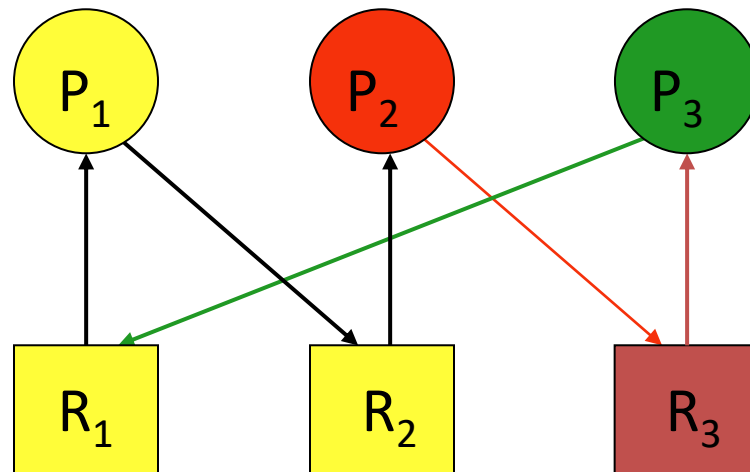
Solution #2: Deadlock Prevention

- Some devices can be managed with a spool
- Example: a printer
 - The spool virtualizes the printer (creates infinite printers)
 - Each process prints on a virtual printer
 - In practice, requests the print to the spool
 - There is no longer mutual exclusion: the physical printer is assigned to the spool process
- Solves the problem, but:
 - Can be applied only in some cases
 - There still can be deadlocks in the buffer used by the spool manager

Solution #2: Deadlock Prevention

Lock ordering

- Requests to resources must be ordered



R_3 : Resource with maximum index

P_3 violates the constraint of sorted requests

P_3 causes circular waiting

Solution #2: Deadlock Prevention

Summary

Condition	Method
Limited access to resources:	Spool / virtualization
“wait while holding”:	Request all resources in advance
Circular waiting:	Lock ordering

Multiplicity of resources

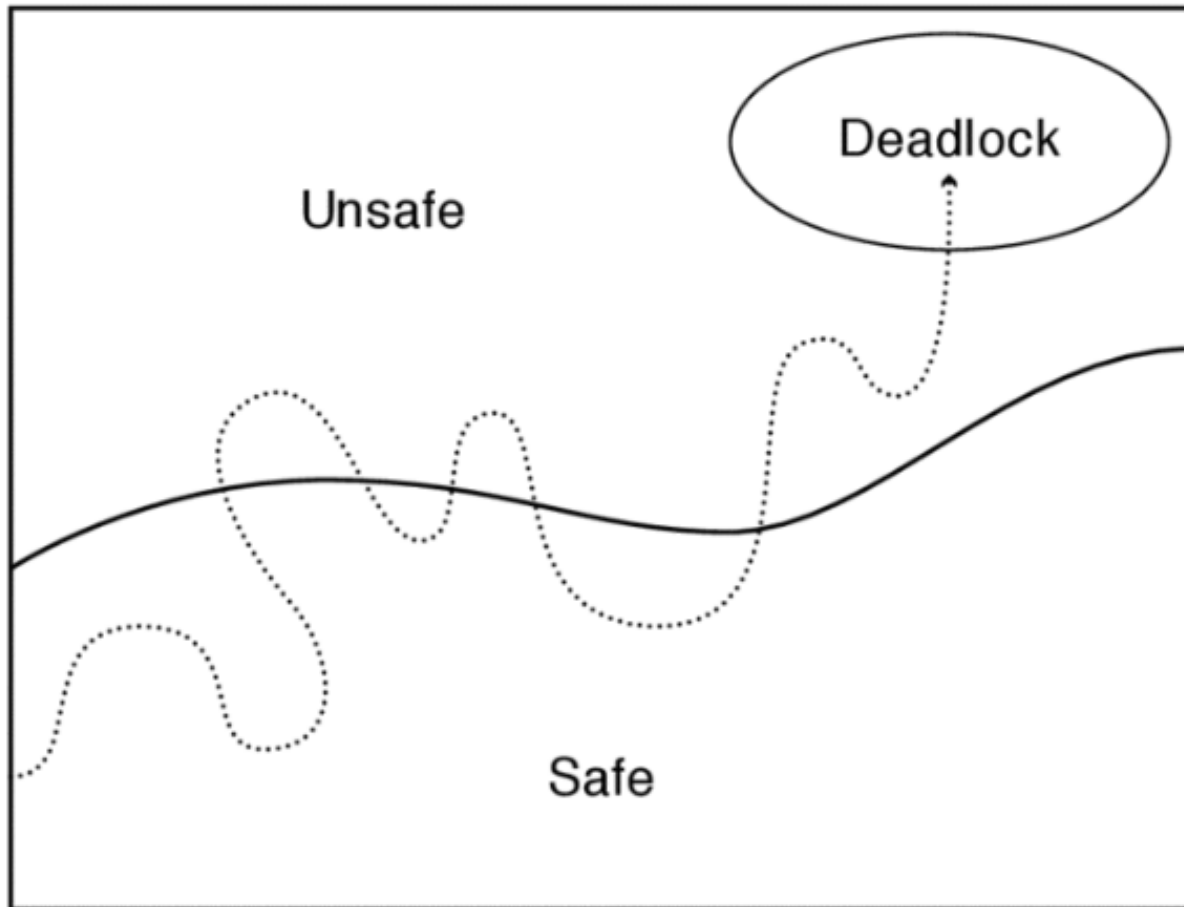
- Resources classified according to their type
 - Multiplicity M :
 - M is the number of resources of a given type
 - Availability D :
 - Number of resources of a given kind that are currently available
- Method of request:
 1. Single resource
 2. Multiple resources
 - A process requests k resources of a given kind
 - If $k \leq D$ then all k resources are assigned
 - If $k > D$ then no resource is assigned (and the process waits)

Solution #3: Banker's Algorithm

- Banker's algorithm
 - State maximum resource needs in advance
 - Allocate resources dynamically when resource is needed -- wait if granting request would lead to deadlock
 - Request can be granted if some sequential ordering of threads is deadlock free

The banker is the resource manager!

Possible System States

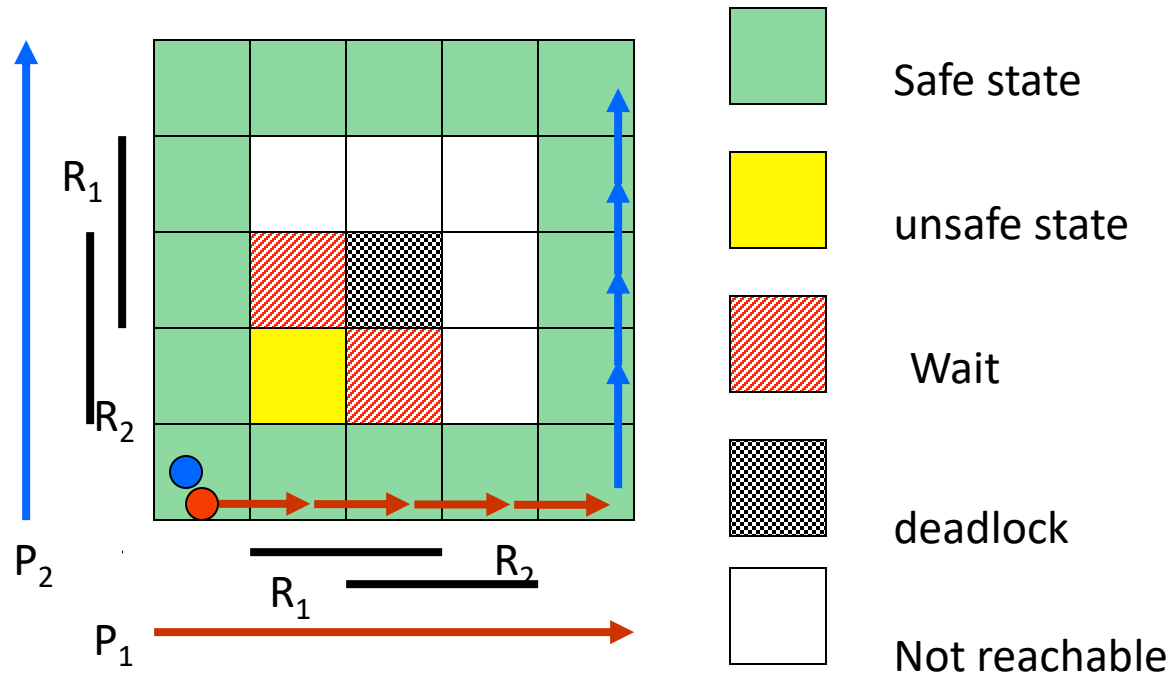


Definitions

- Safe state:
 - For any possible sequence of future resource requests, it is possible to eventually grant all requests*
 - * not necessarily in the same order they are requested...
 - And thus to make all the processes *end* correctly
 - May require waiting even when resources are available!
- Unsafe state:
 - Some sequence of resource requests can result in deadlock
- Doomed state:
 - All possible computations lead to deadlock

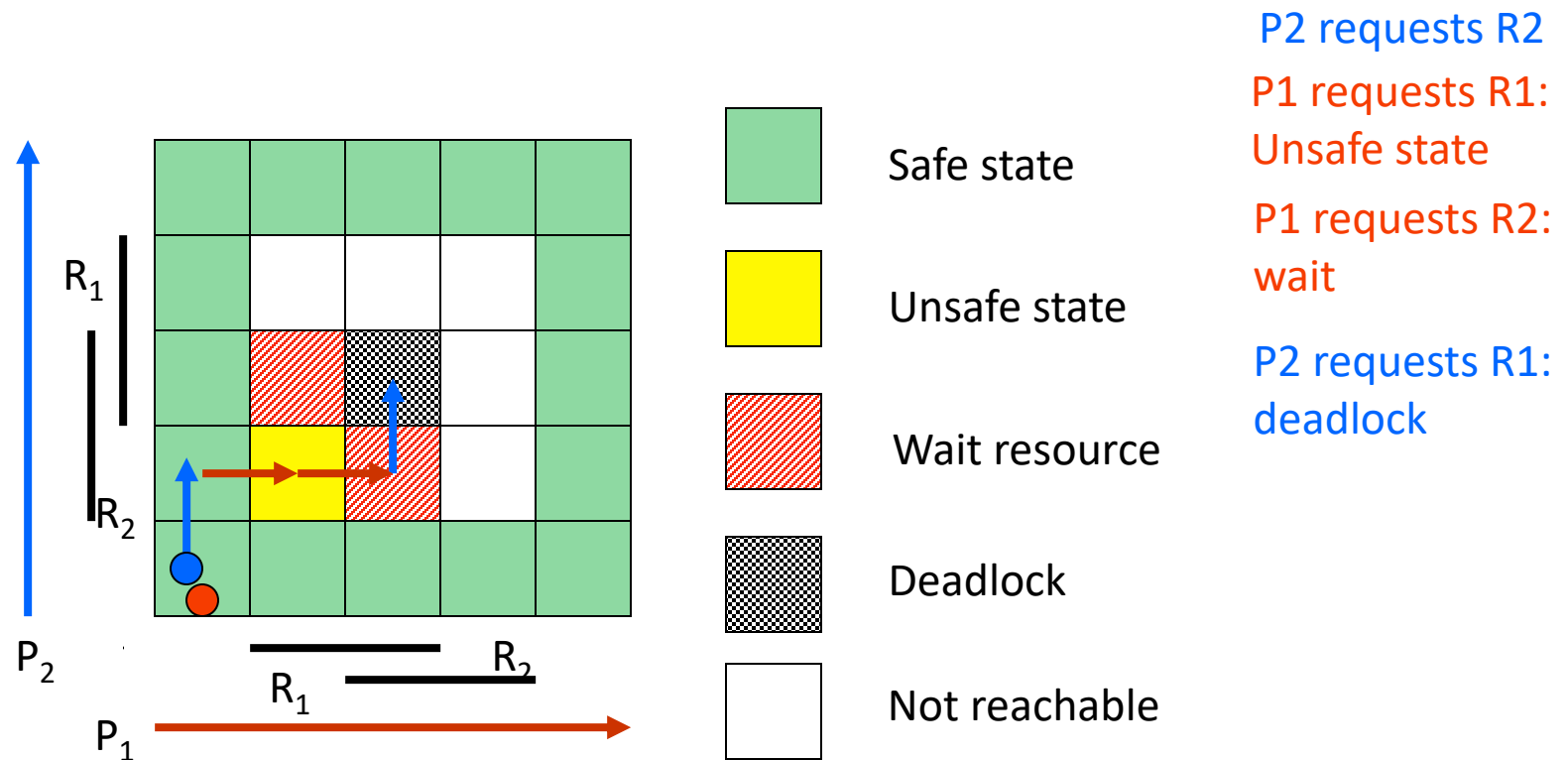
Safe state

1) System that evolves in safe states



Safe state

2) A system in an unsafe state can reach a deadlock



3) The banker does not accept requests that lead to unsafe states



Banker's Algorithm

- Grant request iff(*) result is a safe state
- Sum of maximum resource needs of current threads can be greater than the total resources
 - Provided there is some way for all the threads to end without getting into deadlock
- Example: proceed iff
 - # free resources $\geq$ max remaining that might be needed by this thread in order to finish
 - Guarantees this thread can finish

(*) iff, “if and only if” abbreviation

Banker's Algorithm, resources of the same type

- First each process declares the number of resources it needs
- At a request of process P the banker checks whether the resource assignment keeps a safe state. To this purpose:
 - Considers the state S reached if request is granted
 - For each process computes the residual requirement R (the number of resources it still needs)
 - Sorts the processes for an increasing value of R
 - Executes the algorithm (see next slide)
 - If at all processes are marked at the end of algorithm the state is safe
- If the state S is safe then the request can be granted
- Otherwise the process P waits until there are enough resources to let it proceed

Banker's Algorithm, resources of the multiple types

D : availability vector

For each resource R_k : D_k number of available units of R_k

For each process P_j :

- A_j : assignment vector;
- E_j : vector of residual requirements; ($E_j \leq D$ if $E_{jk} \leq D_k$ for each k)

Initially each process P_j is not marked

```
while ( $\exists$  non marked processes) {  
    if ( $\exists$  a non-marked  $P_j$  that satisfies  $E_j \leq D$ ) {  
        mark  $P_j$ ;  
         $D = D + A_j$  ;  
    } else ends while, the state is not safe;  
}
```

success: the initial state is safe

Banker's Algorithm, Examples

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU'

TIPI (1.1)

Un sistema con processi P1, P2, P3, P4 e risorse dei tipi R1, R2, R3, R4, rispettivamente di molteplicità [4, 5, 5, 5], i processi dichiarano inizialmente le seguenti esigenze:

ESIGENZA INIZIALE				
	R1	R2	R3	R4
P1	2	3	1	1
P2	2	1	1	2
P3	0	1	0	2
P4	0	2	5	2

Stato (**sicuro**) raggiunto dal sistema al tempo t :

ASSEGNAZIONE ATTUALE →

	R1	R2	R3	R4
P1	2	1	1	1
P2	2	0	1	2
P3	0	1	0	0
P4	0	2	2	2

DISPONIBILTA' ATTUALE

R1	R2	R3	R4
0	1	1	0

MOLTEPLICITA' →

R1	R2	R3	R4
4	5	5	5

ESIGENZA RESIDUA

R1	R2	R3	R4
0	2	0	0
0	1	0	0
0	0	0	2
0	0	3	0

VERIFICA: LO STATO RAGGIUNTO AL TEMPO t E' SICURO (Ver 1)

Stato raggiunto dal sistema al tempo t :

ASSEGNAZIONE ATTUALE →	MOLTEPLICITA' →									
		R1	R2	R3	R4		R1	R2	R3	R4
	P1	2	1	1	1		4	5	5	5
	P2	2	0	1	2					
	P3	0	1	0	0					
	P4	0	2	2	2					
DISPONIBILTA' ATTUALE			R1	R2	R3	R4				
			0	1	1	0				
							ESIGENZA RESIDUA			

Verifica dello stato sicuro:

1) l'esigenza di P2 può essere soddisfatta e P2 può terminare

ASSEGNAZIONE ATTUALE →

MOLTEPLICITA' →

	R1	R2	R3	R4
P1	2	1	1	1
P2	-	-	-	-
P3	0	1	0	0
P4	0	2	2	2

DISPONIBILTA' ATTUALE

	R1	R2	R3	R4
	2	1	2	2

ESIGENZA RESIDUA

	R1	R2	R3	R4
	4	5	5	5
P1	0	2	0	0
P2	-	-	-	-
P3	0	0	0	2
P4	0	0	3	0

VERIFICA: LO STATO RAGGIUNTO AL TEMPO t E' SICURO (Ver 2)

Verifica dello stato sicuro:

1) l'esigenza di P2 può essere soddisfatta e P2 può terminare

ASSEGNAZIONE
ATTUALE →

	R1	R2	R3	R4
P1	2	1	1	1
P2	-	-	-	-
P3	0	1	0	0
P4	0	2	2	2

DISPONIBILTA'
ATTUALE

	R1	R2	R3	R4
	2	1	2	2

MOLTEPLICITA' →

	R1	R2	R3	R4
	4	5	5	5

ESIGENZA RESIDUA

	R1	R2	R3	R4
P1	0	2	0	0
P2	-	-	-	-
P3	0	0	0	2
P4	0	0	3	0

Verifica dello stato sicuro:

1) l'esigenza di P2 può essere soddisfatta e P2 può terminare

2) l'esigenza di P3 può essere soddisfatta e P3 può terminare

ASSEGNAZIONE
ATTUALE →

	R1	R2	R3	R4
P1	2	1	1	1
P2	-	-	-	-
P3	-	-	-	-
P4	0	2	2	2

DISPONIBILTA'
ATTUALE

	R1	R2	R3	R4
	2	2	2	2

MOLTEPLICITA' →

	R1	R2	R3	R4
	4	5	5	5

ESIGENZA RESIDUA

	R1	R2	R3	R4
P1	0	2	0	0
P2	-	-	-	-
P3	-	-	-	-
P4	0	0	3	0

VERIFICA: LO STATO RAGGIUNTO AL TEMPO t E' SICURO (Ver 3)

Verifica dello stato sicuro:

- 1) l'esigenza di P2 può essere soddisfatta e P2 può terminare
- 2) l'esigenza di P3 può essere soddisfatta e P3 può terminare

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →									
			R1	R2	R3	R4		R1	R2	R3	R4
		P1	2	1	1	1		4	5	5	5
		P2	-	-	-	-					
		P3	-	-	-	-					
		P4	0	2	2	2					
DISPONIBILTA' ATTUALE			R1	R2	R3	R4					
			2	2	2	2					

Verifica dello stato sicuro:

- 1) l'esigenza di P2 può essere soddisfatta e P2 può terminare
- 2) l'esigenza di P3 può essere soddisfatta e P3 può terminare
- 3) l'esigenza di P1 può essere soddisfatta e P1 può terminare

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →										
			R1	R2	R3	R4		R1	R2	R3	R4	
		P1	-	-	-	-		4	5	5	5	
		P2	-	-	-	-						
		P3	-	-	-	-		P1	-	-	-	
		P4	0	2	2	2		P2	-	-	-	
DISPONIBILTA' ATTUALE			R1	R2	R3	R4						
			4	3	3	3						
								P3	-	-	-	
								P4	0	0	3	0
						ESIGENZA RESIDUA						

VERIFICA: LO STATO RAGGIUNTO AL TEMPO t E' SICURO (Ver 4)

Verifica dello stato sicuro:

- 1) l'esigenza di P2 può essere soddisfatta e P2 può terminare
- 2) l'esigenza di P3 può essere soddisfatta e P3 può terminare
- 3) l'esigenza di P1 può essere soddisfatta e P1 può terminare

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →									
			R1	R2	R3	R4		R1	R2	R3	R4
		P1	-	-	-	-		4	5	5	5
		P2	-	-	-	-					
		P3	-	-	-	-					
		P4	0	2	2	2					
DISPONIBILTA' ATTUALE			R1	R2	R3	R4					
			4	3	3	3					
		P1	-	-	-	-					
		P2	-	-	-	-					
		P3	-	-	-	-					
		P4	0	0	3	0	ESIGENZA RESIDUA				

Verifica dello stato sicuro:

- 1) l'esigenza di P2 può essere soddisfatta e P2 può terminare
- 2) l'esigenza di P3 può essere soddisfatta e P3 può terminare
- 3) l'esigenza di P1 può essere soddisfatta e P1 può terminare
- 4) l'esigenza di P4 può essere soddisfatta e P4 può terminare

STATO SICURO !

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →									
			R1	R2	R3	R4		R1	R2	R3	R4
		P1	-	-	-	-		4	5	5	5
		P2	-	-	-	-					
		P3	-	-	-	-					
		P4	-	-	-	-					
DISPONIBILTA' ATTUALE			R1	R2	R3	R4					
			4	5	5	5					
		P1	-	-	-	-					
		P2	-	-	-	-					
		P3	-	-	-	-					
		P4	-	-	-	-	ESIGENZA RESIDUA				

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (1.2)

Stato (**sicuro**) raggiunto dal sistema al tempo t :

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →									
			R1	R2	R3	R4		R1	R2	R3	R4
		P1	2	1	1	1	4	5	5	5	
		P2	2	0	1	2					
		P3	0	1	0	0	P1	0	2	0	0
		P4	0	2	2	2	P2	0	1	0	0
DISPONIBILTA' ATTUALE			R1	R2	R3	R4	P3	0	0	0	2
		0	1	1	0	P4	0	0	3	0	
							ESIGENZA RESIDUA				

Il processo P1 richiede una risorsa di tipo R2: stato raggiunto dopo l'ipotetica assegnazione:

		MOLTEPLICITA' →								
		R1	R2	R3	R4	R1	R2	R3	R4	
		4	5	5	5					
ASSEGNAZIONE ATTUALE →		R1	R2	R3	R4		R1	R2	R3	R4
	P1	2	2	1	1	P1	0	1	0	0
	P2	2	0	1	2	P2	0	1	0	0
	P3	0	1	0	0	P3	0	0	0	2
	P4	0	2	2	2	P4	0	0	3	0
DISPONIBILTA' ATTUALE		R1	R2	R3	R4	ESIGENZA RESIDUA				
		0	0	1	0					

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (1.3)

Il processo P1 richiede una risorsa di tipo R2: stato raggiunto dopo l'ipotetica assegnazione:

		MOLTEPLICITA' →								
		R1	R2	R3	R4					
		4	5	5	5					
ASSEGNAZIONE ATTUALE →	P1	2	2	1	1					
	P2	2	0	1	2					
	P3	0	1	0	0					
	P4	0	2	2	2					
		R1	R2	R3	R4					
		0	0	1	0					
DISPONIBILTA' ATTUALE						ESIGENZA RESIDUA				
		R1	R2	R3	R4	P1	0	1	0	0
						P2	0	1	0	0
						P3	0	0	0	2
						P4	0	0	3	0

Verifica dello stato sicuro:

- 1) l'esigenza di P1 non può essere soddisfatta
- 2) l'esigenza di P2 non può essere soddisfatta
- 3) l'esigenza di P3 non può essere soddisfatta
- 4) l'esigenza di P4 non può essere soddisfatta

LO STATO NON E' SICURO, LA RICHIESTA INIZIALE DI P1 NON PUO' ESSERE SODDISFATTA!

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (2.1)

Un sistema con processi P1, P2, P3, P4 e risorse dei tipi R1, R2, R3, R4, rispettivamente di molteplicità [4, 5, 5, 5], i processi dichiarano inizialmente le seguenti esigenze:

ESIGENZA INIZIALE				
	R1	R2	R3	R4
P1	2	3	1	1
P2	2	0	1	3
P3	0	1	0	2
P4	0	2	3	2

Stato raggiunto dal sistema al tempo t :

ASSEGNAZIONE ATTUALE →	MOLTEPLICITA' →					R1	R2	R3	R4
		R1	R2	R3	R4	4	5	5	5
	P1	2	1	1	1				
	P2	2	0	1	2				
	P3	0	1	0	0				
	P4	0	2	2	2				
DISPONIBILTA' ATTUALE		R1	R2	R3	R4				
		0	1	1	0				

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (2.2)

Stato raggiunto dal sistema al tempo t :

ASSEGNAZIONE ATTUALE →	MOLTEPLICITA' →									
		R1	R2	R3	R4		R1	R2	R3	R4
	P1	2	1	1	1		4	5	5	5
	P2	2	0	1	2					
	P3	0	1	0	0					
	P4	0	2	2	2					

Al tempo t processo P1 richiede una risorsa di tipo R2: stato raggiunto dopo l'ipotetica assegnazione:

ASSEGNAZIONE ATTUALE →	MOLTEPLICITA' →									
		R1	R2	R3	R4		R1	R2	R3	R4
	P1	2	2	1	1		4	5	5	5
	P2	2	0	1	2					
	P3	0	1	0	0		P1	0	1	0
	P4	0	2	2	2		P2	0	0	0
							P3	0	0	0
							P4	0	0	1
				</						

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (2.3)

Al tempo t processo P1 richiede una risorsa di tipo R2: stato raggiunto dopo l'ipotetica assegnazione:

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →										
			R1	R2	R3	R4		R1	R2	R3	R4	
		P1	2	2	1	1		4	5	5	5	
		P2	2	0	1	2						
		P3	0	1	0	0		P1	0	1	0	0
		P4	0	2	2	2		P2	0	0	0	1
DISPONIBILTA' ATTUALE			R1	R2	R3	R4		P3	0	0	0	2
			0	0	1	0		P4	0	0	1	0
							ESIGENZA RESIDUA					

Verifica dello stato sicuro:

1) l'esigenza di P4 può essere soddisfatta e P4 può terminare

ASSEGNAZIONE ATTUALE →

MOLTEPLICITA' →

	R1	R2	R3	R4
P1	2	2	1	1
P2	2	0	1	2
P3	0	1	0	0
P4	-	-	-	-

DISPONIBILTA' ATTUALE

	R1	R2	R3	R4
	0	2	3	2

ESIGENZA RESIDUA

	R1	R2	R3	R4
	4	5	5	5
P1	0	1	0	0
P2	0	0	0	1
P3	0	0	0	2
P4	-	-	-	-

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (2.4)

Verifica dello stato sicuro:

1) l'esigenza di P4 può essere soddisfatta e P4 può terminare

ASSEGNAZIONE
ATTUALE →

	R1	R2	R3	R4
P1	2	2	1	1
P2	2	0	1	2
P3	0	1	0	0
P4	-	-	-	-

DISPONIBILTA'
ATTUALE

	R1	R2	R3	R4
	0	2	3	2

MOLTEPLICITA' →

	R1	R2	R3	R4
	4	5	5	5

ESIGENZA RESIDUA

	R1	R2	R3	R4
P1	0	1	0	0
P2	0	0	0	1
P3	0	0	0	2
P4	-	-	-	-

Verifica dello stato sicuro:

1) l'esigenza di P4 può essere soddisfatta e P4 può terminare

2) l'esigenza di P1 può essere soddisfatta e P1 può terminare

ASSEGNAZIONE
ATTUALE →

	R1	R2	R3	R4
P1	-	-	-	-
P2	2	0	1	2
P3	0	1	0	0
P4	-	-	-	-

MOLTEPLICITA' →

	R1	R2	R3	R4
	4	5	5	5

DISPONIBILTA'
ATTUALE

	R1	R2	R3	R4
	2	4	4	3

ESIGENZA RESIDUA

	R1	R2	R3	R4
P1	-	-	-	-
P2	0	0	0	1
P3	0	0	0	2
P4	-	-	-	-

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (2.5)

Verifica dello stato sicuro:

- 1) l'esigenza di P4 può essere soddisfatta e P4 può terminare
- 2) l'esigenza di P1 può essere soddisfatta e P1 può terminare

		MOLTEPLICITA' →							
		R1	R2	R3	R4	R1	R2	R3	R4
ASSEGNAZIONE ATTUALE →	P1	-	-	-	-	4	5	5	5
	P2	2	0	1	2				
	P3	0	1	0	0				
	P4	-	-	-	-				
		DISPONIBILTA' ATTUALE							
		R1	R2	R3	R4				
		2	4	4	3				
						ESIGENZA RESIDUA			

Verifica dello stato sicuro:

- 1) l'esigenza di P4 può essere soddisfatta e P4 può terminare
- 2) l'esigenza di P1 può essere soddisfatta e P1 può terminare
- 3) l'esigenza di P2 può essere soddisfatta e P2 può terminare

		MOLTEPLICITA' →							
		R1	R2	R3	R4	R1	R2	R3	R4
ASSEGNAZIONE ATTUALE →	P1	-	-	-	-	4	5	5	5
	P2	-	-	-	-				
	P3	0	1	0	0				
	P4	-	-	-	-				
		DISPONIBILTA' ATTUALE							
		R1	R2	R3	R4				
		4	4	5	5				
						ESIGENZA RESIDUA			

ALGORITMO DEL BANCHIERE CON RISORSE MULTIPLE DI PIU' TIPI (2.6)

Verifica dello stato sicuro:

- 1) l'esigenza di P4 può essere soddisfatta e P4 può terminare
- 2) l'esigenza di P1 può essere soddisfatta e P1 può terminare
- 3) l'esigenza di P2 può essere soddisfatta e P2 può terminare

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →									
			R1	R2	R3	R4		R1	R2	R3	R4
		P1	-	-	-	-		4	5	5	5
		P2	-	-	-	-					
		P3	0	1	0	0		P1	-	-	-
		P4	-	-	-	-		P2	-	-	-
DISPONIBILTA' ATTUALE			R1	R2	R3	R4		P3	0	0	0
			4	4	5	5		P4	-	-	-
						ESIGENZA RESIDUA					

Verifica dello stato sicuro:

- 1) l'esigenza di P4 può essere soddisfatta e P4 può terminare
- 2) l'esigenza di P1 può essere soddisfatta e P1 può terminare
- 3) l'esigenza di P2 può essere soddisfatta e P2 può terminare
- 4) l'esigenza di P3 può essere soddisfatta e P3 può terminare

**STATO SICURO ! LA RICHIESTA INIZIALE
DI P1 PUO' ESSERE SODDISFATTA.**

ASSEGNAZIONE ATTUALE →		MOLTEPLICITA' →									
			R1	R2	R3	R4		R1	R2	R3	R4
		P1	-	-	-	-		4	5	5	5
		P2	-	-	-	-					
		P3	-	-	-	-		P1	-	-	-
		P4	-	-	-	-		P2	-	-	-
DISPONIBILTA' ATTUALE			R1	R2	R3	R4		P3	-	-	-
			4	5	5	5		P4	-	-	-
						ESIGENZA RESIDUA					